FINAL PROJECT

202322621

Individual Assignment

Phase 3

Selitse M.S

1. Architecture

Architectural style: strict layered architecture

The system uses a **strict four-layer architecture**: Presentation, Business logic, Data access, and External libraries and storage. Each layer may only call the layer directly below it, and only through interfaces. Domain objects (User, Assignment, Submission and so on) are plain C++ classes that are passed between the layers. The style was chosen because it answers specific requirements rather than because it is common:

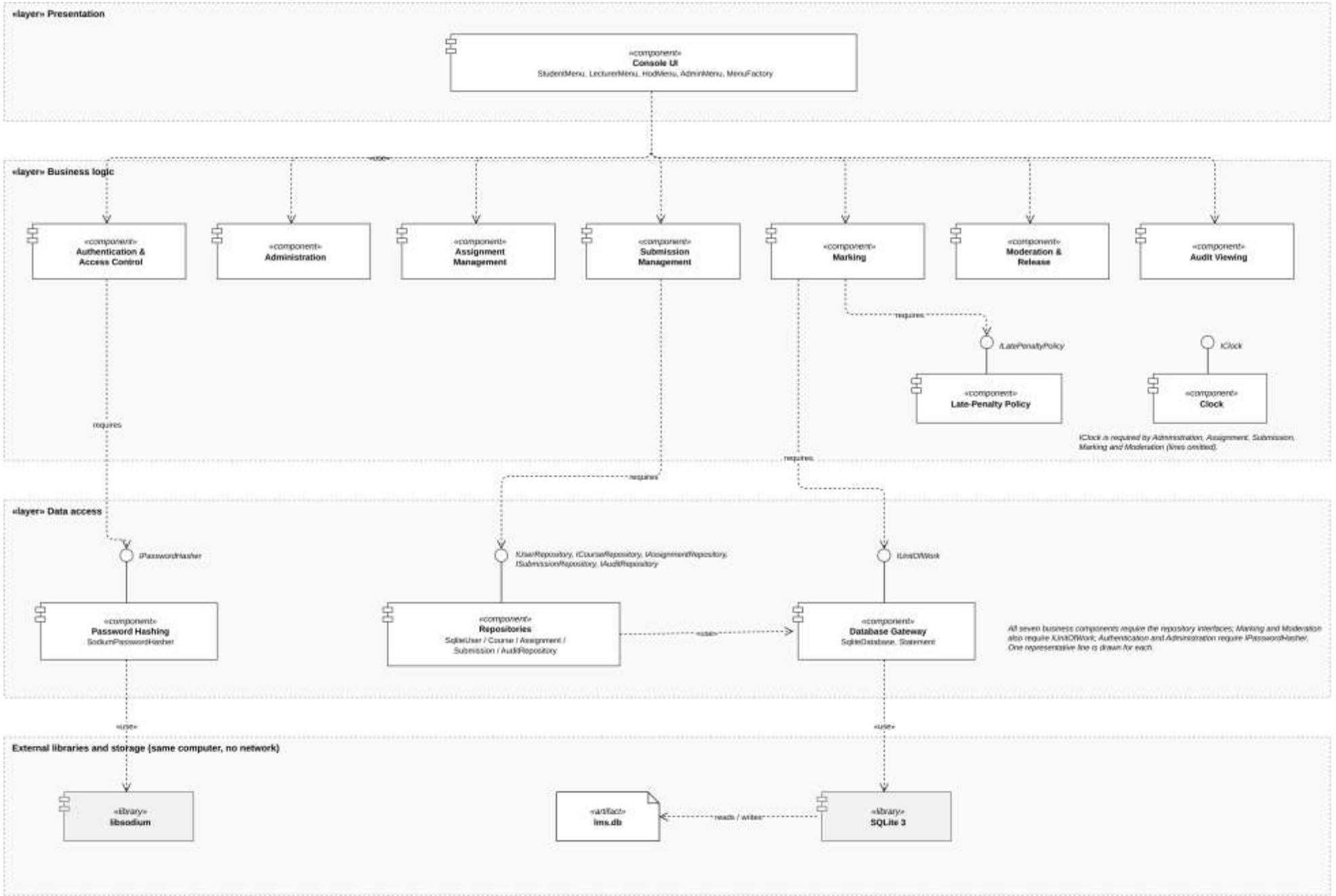
- **Testability.** The brief requires at least 70% line coverage of the business-logic layer. With strict layering that layer has no console input/output and no SQL, so it can be unit-tested on its own by injecting an in-memory SQLite database and a fixed clock.
- **Offline, single machine (NFR03).** The only storage is an embedded SQLite file reached through the data-access layer. There is no server process and no network dependency.
- **Security (FR20, FR21, NFR04, NFR06).** Every permission check lives in one place in the business layer (Session::require), so a menu cannot skip it. All SQL lives in the data layer and uses parameterised statements only.
- **Change (CR1, Phase 8).** A class-wide extension is expected to touch Assignment Service, the assignment repository and the extension table, plus one new menu option. The rest of the layers are unaffected.
- **Simplicity for non-IT staff (C4, NFR02).** The presentation layer shows each role a short numbered menu containing only that role's actions.

Component diagram

The table shows the components in each layer. Provided interfaces are drawn as circles ("lollipops"); a dashed arrow to a circle means the component requires that interface. Table 5.1 states each component's responsibility and the requirements it serves.

Component	Layer	Responsibility	Requirements
Console UI	Presentation	Reads input, shows one menu per role, catches every LmsError and prints a clear message	FR20, NFR02
Authentication & Access Control	Business logic	Verifies the password hash at login; Session::require() enforces role permissions	FR20, FR21, NFR04
Administration	Business logic	User accounts, departments, courses, lecturer assignment, enrolment	FR01 to FR04
Assignment Management	Business logic	Creates assignments, grants individual extensions, lists courses and assignments	FR05, FR06, FR14, FR15, FR23
Submission Management	Business logic	Submission, resubmission, late detection, student results	FR13, FR16, FR17, FR19
Marking	Business logic	Records mark and feedback, applies the late penalty, writes the audit entry	FR07 to FR09, FR18, FR22
Moderation & Release	Business logic	HoD review, approval or adjustment with a reason, release	FR10 to FR12, FR22
Audit Viewing	Business logic	Shows the change history of a mark to the HoD and Administrator	FR22, O1

Component	Layer	Responsibility	Requirements
Late-Penalty Policy	Business logic	Strategy object holding the penalty rule	FR18, C2
Clock	Business logic	Supplies the current time; replaced by a fixed clock in tests	FR16, FR18
Repositories	Data access	All SQL for each group of tables, parameterised	NFR01, NFR05
Database Gateway	Data access	Connection, prepared statements, transactions	NFR03, FR22
Password Hashing	Data access	Argon2id hashing through libsodium	NFR04


«layer» Presentation
«component»
Console UI
StudentMenu, LecturerMenu, HodMenu, AdminMenu, MenuFactory
«use»
«layer» Business logic
«component»
Authentication & Access Control
«component»
Administration
«component»
Assignment Management
«component»
Submission Management
«component»
Marking
«component»
Moderation & Release
«component»
Audit Viewing
requires
ILatePenaltyPolicy
«component»
Late-Penalty Policy
IClock
«component»
Clock
IClock is required by Administration, Assignment, Submission, Marking and Moderation (lines omitted).
requires
requires
requires
«layer» Data access
IPasswordHasher
«component»
Password Hashing
SodiumPasswordHasher
IUserRepository, ICourseRepository, IAssignmentRepository, ISubmissionRepository, IAuditRepository
«component»
Repositories
SqliteUser / Course / Assignment / Submission / AuditRepository
«use»
IUnitOfWork
«component»
Database Gateway
SqliteDatabase, Statement
All seven business components require the repository interfaces; Marking and Moderation also require IUnitOfWork; Authentication and Administration require IPasswordHasher. One representative line is drawn for each.
«use»
«use»
External libraries and storage (same computer, no network)
«library»
libsodium
«artifact»
ims.db
reads / writes
«library»
SQLite 3

Design rationale: alternatives considered and rejected

Alternative	Why it was considered	Why it was rejected
Model-View-Controller (MVC)	A common way to separate the user interface from the data.	MVC earns its keep when several views must stay in sync with a changing model (usually through Observer). A console program shows one screen at a time, so there is nothing to keep in sync. MVC also does not say where database code belongs, so controllers tend to collect both business rules and SQL. The layered style draws a clear boundary around the business-logic layer, which is exactly the unit the brief asks us to measure test coverage on.
Client-server with a central database server (e.g. MySQL)	Several computers could share one database.	Needs a running server and a reliable network. The IT officer states that the computers are old and the internet is unreliable (NFR03). This was already rejected in the feasibility study (Section 3.1), and the design stays consistent with it.
Relaxed layering (menus may read repositories directly)	Avoids a few "pass-through" service methods.	A student menu could then read the submission table directly and bypass the rule that a mark stays hidden until it is released (FR13, C1). A few extra service methods are a small price for the guarantee that every read passes through a permission check.
Singleton database connection	Easy global access to the database.	Hidden global state makes unit tests share one database and fail depending on the order they run in. Instead, main() creates the connection once and passes it to the repositories through their constructors (dependency injection).

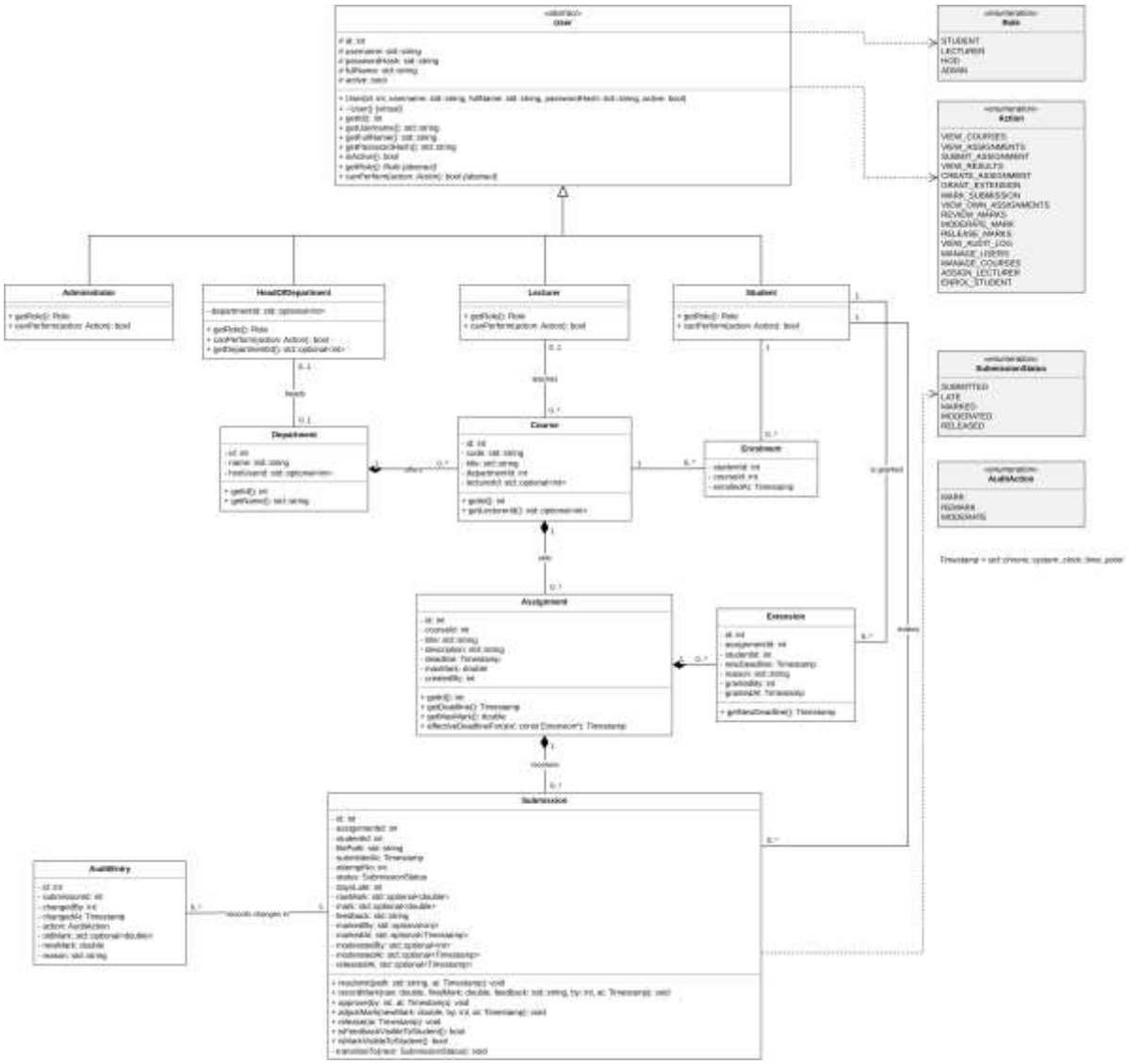
2. Class Diagram

Domain Model

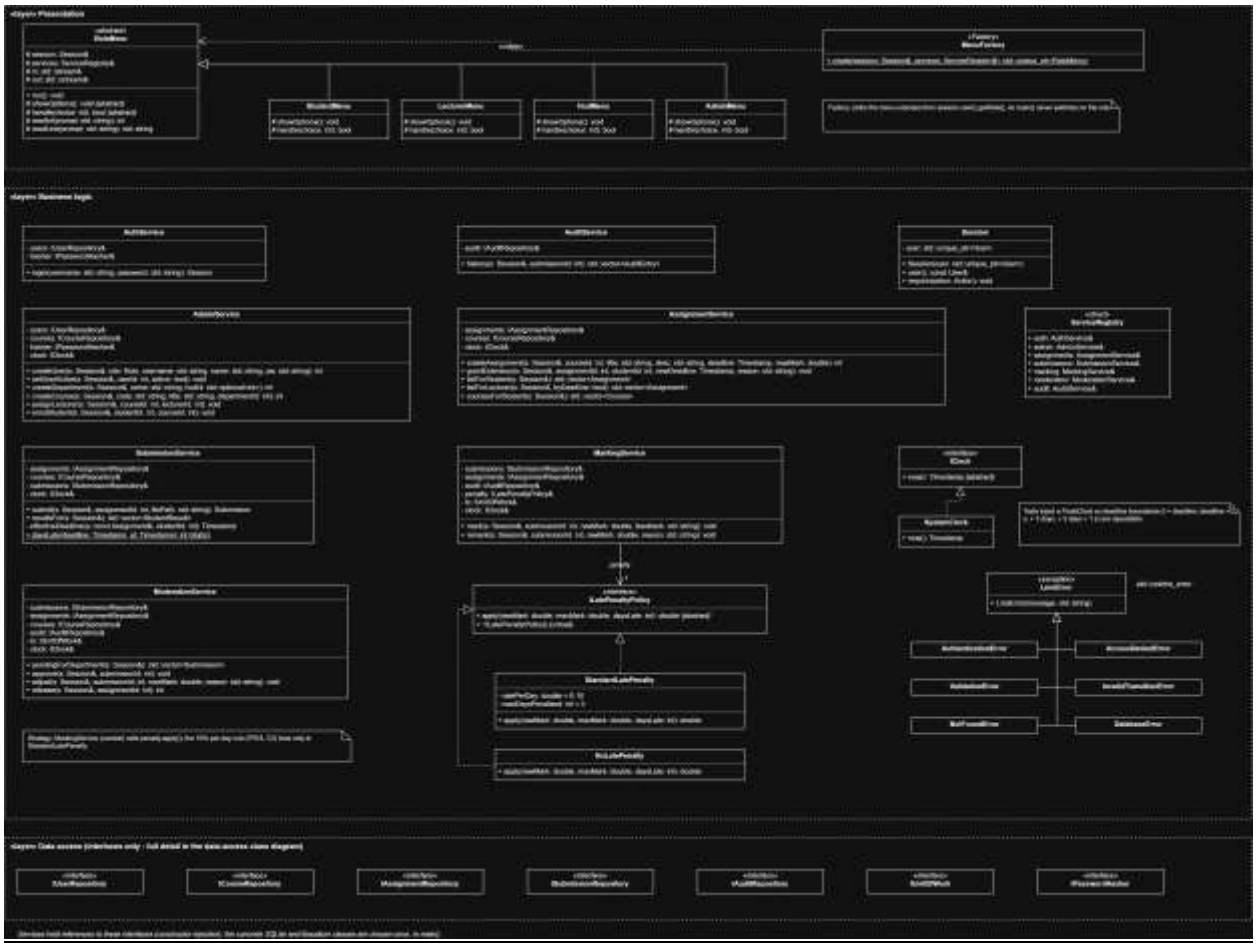
Role class	Permitted actions (canPerform returns true)
Student	VIEW_COURSES, VIEW_ASSIGNMENTS, SUBMIT_ASSIGNMENT, VIEW_RESULTS
Lecturer	CREATE_ASSIGNMENT, GRANT_EXTENSION, MARK_SUBMISSION, VIEW_OWN_ASSIGNMENTS
HeadOfDepartment	REVIEW_MARKS, MODERATE_MARK, RELEASE_MARKS, VIEW_AUDIT_LOG
Administrator	MANAGE_USERS, MANAGE_COURSES, ASSIGN_LECTURER, ENROL_STUDENT, VIEW_AUDIT_LOG

Submission owns its own life-cycle rules. Every status change goes through the private operation transitionTo(), which checks the state machine in Figure 5.5 and throws InvalidTransitionError for anything not allowed. isMarkVisibleToStudent() returns true only in the Released state, and isFeedbackVisibleToStudent() returns true from Marked onwards. These two operations implement the C1 resolution. Composition (filled diamond) is used where the part cannot exist without the whole: a course owns its assignments, and an assignment owns its submissions and extensions.

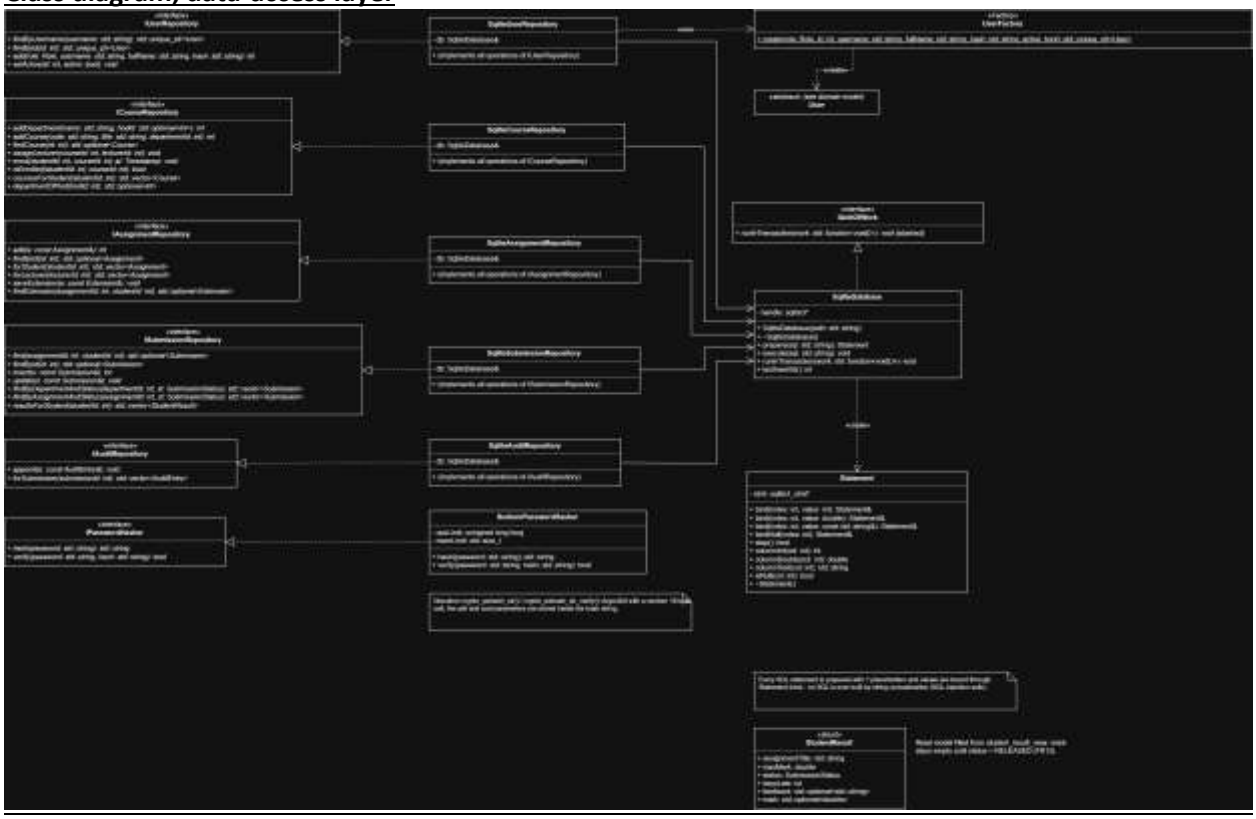
Class diagram, domain model



Class diagram, presentation and business-logic layers


«layer» Presentation
«abstract»
BaseMenu
session: Session
services: ServiceRegistry
in: Scanner
out: PrintStream
+ run(): void
showOptions(): void
handle(choice: int): bool
readNonEmpty(prompt: string): string
readLong(prompt: string): long
MainMenu
showOptions(): void
handle(choice: int): bool
CourseMenu
showOptions(): void
handle(choice: int): bool
GradeMenu
showOptions(): void
handle(choice: int): bool
AdminMenu
showOptions(): void
handle(choice: int): bool
«Factory»
MenuFactory
+ create(session: Session, services: ServiceRegistry, in: Scanner, out: PrintStream): MainMenu
Factory picks the menu subclass from session.user.getRole(), so main() never switches on the role.
«layer» Business logic
AuthService
- users: UserRepository
- hasher: PasswordHasher
+ login(username: string, password: string): Session
AuditService
- audit: AuditRepository
+ log(s: Session, action: string, target: string, details: string): void
Session
- user: User
- unique_id: Uuid
- created_at: DateTime
+ isExpired(now: Timestamp): bool
+ touch(now: Timestamp): void
AdminService
- users: UserRepository
- courses: CourseRepository
- terms: TermRepository
- clock: Clock
+ createUser(s: Session, role: Role, username: string, name: string, pw: string): User
+ setStatus(s: Session, userId: int, active: bool): void
+ createDepartment(s: Session, name: string): Department
+ createCourse(s: Session, code: string, title: string, deptId: int): Course
+ assignLecturer(s: Session, courseId: int, lecturerId: int): void
+ enrollStudent(s: Session, studentId: int, courseId: int): void
AssignmentService
- assignments: AssignmentRepository
- courses: CourseRepository
- clock: Clock
+ createAssignment(s: Session, courseId: int, title: string, desc: string, deadline: Timestamp, maxMark: double): int
+ publish(session: Session, assignmentId: int, studentId: int, newDeadline: Timestamp, reason: string): void
+ delete(s: Session, id: int): void
+ listForCourse(s: Session, courseId: int): vector<Assignment>
+ current(s: Session): int
«struct»
ServiceRegistry
+ auth: AuthService&
+ admin: AdminService&
+ assignments: AssignmentService&
+ submissions: SubmissionService&
+ marking: MarkingService&
+ reminders: ReminderService&
+ audit: AuditService&
SubmissionService
- assignments: AssignmentRepository
- courses: CourseRepository
- submissions: SubmissionRepository
- clock: Clock
+ submit(s: Session, assignmentId: int, file: string): Submission
+ resubmit(s: Session, id: int): Submission
+ effectiveDeadline(a: const Assignment&, student: int): Timestamp
+ daysLate(deadline, submitted: Timestamp): int
MarkingService
- submissions: SubmissionRepository
- assignments: AssignmentRepository
- audit: AuditService&
- penalty: LatePenaltyPolicy
- in: ScriptReader
- clock: Clock
+ mark(s: Session, submissionId: int, rawMark: double, feedback: string): void
+ override(s: Session, submissionId: int, newMark: double, reason: string): void
«interface»
Clock
+ now(): Timestamp
SystemClock
+ now(): Timestamp
Tests inject a FixedClock so deadline boundaries (t == deadline, deadline + 1 s) are deterministic.
«interface»
LatePenaltyPolicy
+ apply(rawMark: double, daysLate: int): double {pure}
+ id(): string {pure}
«interface»
LineSource
+ readLine(prompt: string): string
std::cin/std::cout
AuthenticationError
AccessDeniedError
NotFoundError
InvalidTransitionError
ValidationError
DatabaseError
StandardLatePolicy
- ratePerDay: double = 0.10
- maxDaysPenalised: int = 5
+ apply(rawMark: double, rawMark: double, daysLate: int): double
ExcuseLatePolicy
+ apply(rawMark: double, rawMark: double, daysLate: int): double
Strategy: ExcuseLatePolicy zeroes late penalty; apply() for 10% per day via STRICT_LATE_POLICY only in StandardLatePolicy.
«layer» Data access (interfaces only - full detail in the data-access class diagram)
«interface»
IUserRepository
«interface»
ICourseRepository
«interface»
IAssignmentRepository
«interface»
ISubmissionRepository
«interface»
IAuditRepository
«struct»
SqlDBRef
«interface»
IPasswordHasher
Services hold references to these interfaces (constructor-injected). The concrete SQLite and Postgres classes are chosen once, in main().

Class diagram, data-access layer



Design patterns

Pattern	Where it is applied	Problem it solves	Why this pattern, and what was rejected
Strategy	ILatePenaltyPolicy with StandardLatePenalty and NoLatePenalty; context: MarkingService	The late-penalty rule was the most disputed requirement (C2) and is school policy that may change.	The rule lives in one small class that can be replaced without editing MarkingService (Open/Closed) and can be tested on its own with boundary values. Rejected: an if/else chain inside MarkingService, which mixes policy with workflow.
Repository	IUserRepository, ICourseRepository, IAssignmentRepository, ISubmissionRepository, IAuditRepository and their SQLite classes	Business rules must not depend on SQL, and tests need a database they can control.	All SQL sits in one layer, and services depend only on interfaces. Rejected: Active Record (e.g. Submission::save()), which puts SQL into the domain classes and makes them impossible to test without a database.
Factory	UserFactory (data access) and MenuFactory (presentation)	A row from the users table holds the role as text and must become the correct User subclass; after login the matching menu must be built.	The decision based on role is written exactly once in each place. Adding a role means one new subclass and one new case. Rejected: repeating a switch on the role in every caller.

Patterns considered and deliberately not used. The **State** pattern (one class per submission state) was rejected: there are only five states with simple transitions, so a transition check inside Submission is

shorter and avoids five classes that would carry almost no behaviour. **Observer** was considered for audit logging and rejected. FR22 requires an audit entry for every mark change, and that entry must be written in the same transaction as the change. An observer that could be detached, or could fail separately, would risk a changed mark with no audit record.

Worked example of StandardLatePenalty (maximum mark 50, raw mark 40). The rule is mark = max(0, raw - 0.10 x maxMark x daysLate) for 1 to 3 days late, and 0 beyond three days:

Days late	**0**	**1**	**2**	**3**	**4 or more**
Deduction	0	5	10	15	all
Final mark	40	35	30	25	0

SOLID principles

Principle	**How the design follows it**
Single responsibility	Each class has one reason to change: the marking workflow in MarkingService, the penalty rule in StandardLatePenalty, SQL in the repositories, hashing in SodiumPasswordHasher, input/output in the menus.
Open/closed	A new penalty rule or a new role is added as a new class, not by editing existing ones. NoLatePenalty was added without touching MarkingService.
Liskov substitution	Any User subclass can be used wherever a User is expected (for example in Session), and none of them strengthens the preconditions of canPerform(). Any repository implementation, whether SQLite or an in-memory test fake, can replace another.
Interface segregation	Five small repository interfaces instead of one large "IDatabase"; IClock and IUnitOfWork have one operation each. MarkingService never sees user-management operations.
Dependency inversion	Services depend only on abstractions passed in through their constructors. Only main() (the composition root) knows the concrete classes SqliteDatabase, SodiumPasswordHasher and SystemClock.

3. Behavioural Models

State machine: Submission life cycle

Four decisions are worth highlighting:

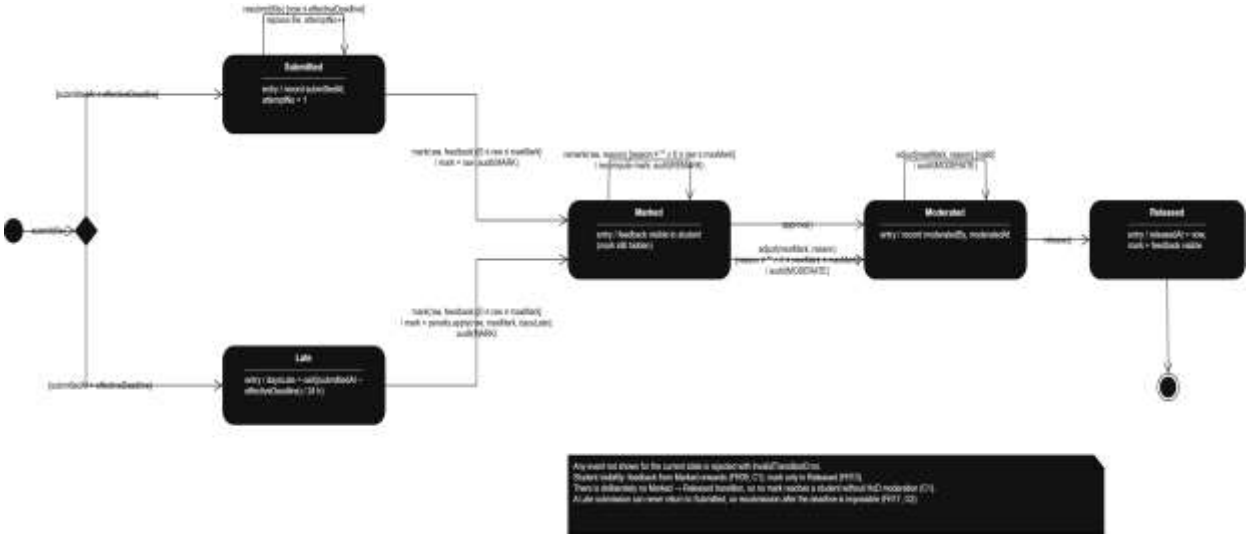
- **Late** is a separate state rather than a flag, because both the penalty rule and the resubmission rule are different for late work.
- There is **no Marked to Released transition**: a mark cannot reach a student without HoD moderation. This is how the C1 conflict is resolved in the design.
- Self-transitions (remark, adjust) always write an audit entry (FR22).
- Any other event raises InvalidTransitionError. The same rules are enforced a second time by CHECK constraints in the database, so the life cycle is protected even if the code has a bug.

Submission state transition table

From	**Event**	**Guard**	**To**	**Action**
(initial)	submit(file)	t <= effective deadline	Submitted	record submittedAt; attemptNo = 1

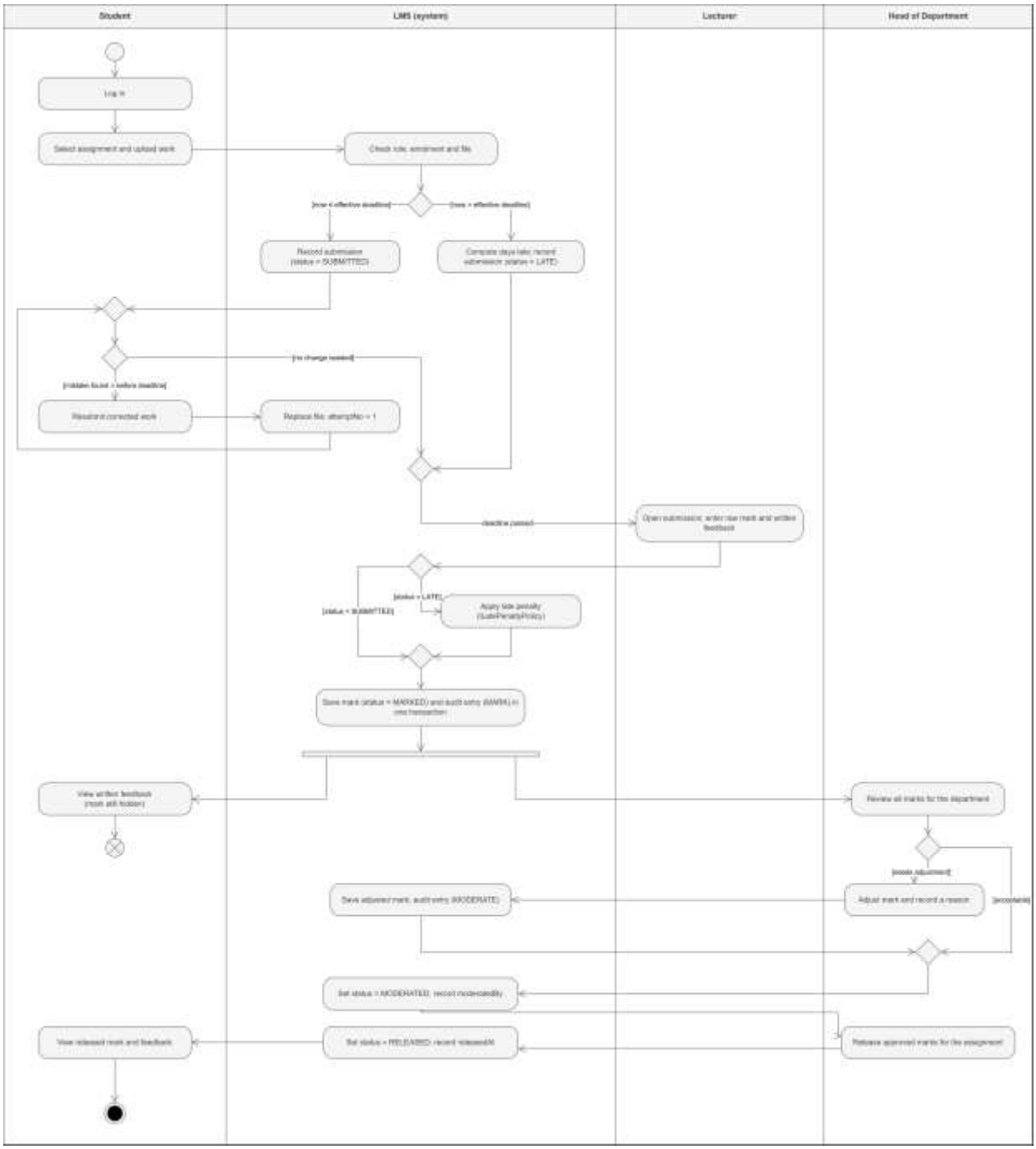
From	Event	Guard	To	Action
(initial)	submit(file)	t > effective deadline	Late	daysLate = ceil((t - deadline) / 24 h)
Submitted	resubmit(file)	now <= effective deadline	Submitted	replace file; attemptNo + 1
Submitted	mark(raw, feedback)	0 <= raw <= maxMark	Marked	mark = raw; audit MARK
Late	mark(raw, feedback)	0 <= raw <= maxMark	Marked	mark = penalty.apply(...); audit MARK
Marked	remark(raw, reason)	reason not empty, raw valid	Marked	recompute mark; audit REMARK
Marked	approve()	none	Moderated	record moderatedBy
Marked	adjust(newMark, reason)	reason not empty, mark valid	Moderated	audit MODERATE
Moderated	adjust(newMark, reason)	reason not empty, mark valid	Moderated	audit MODERATE
Moderated	release()	none	Released	releasedAt = now

State machine diagram, Submission life cycle



Activity diagram

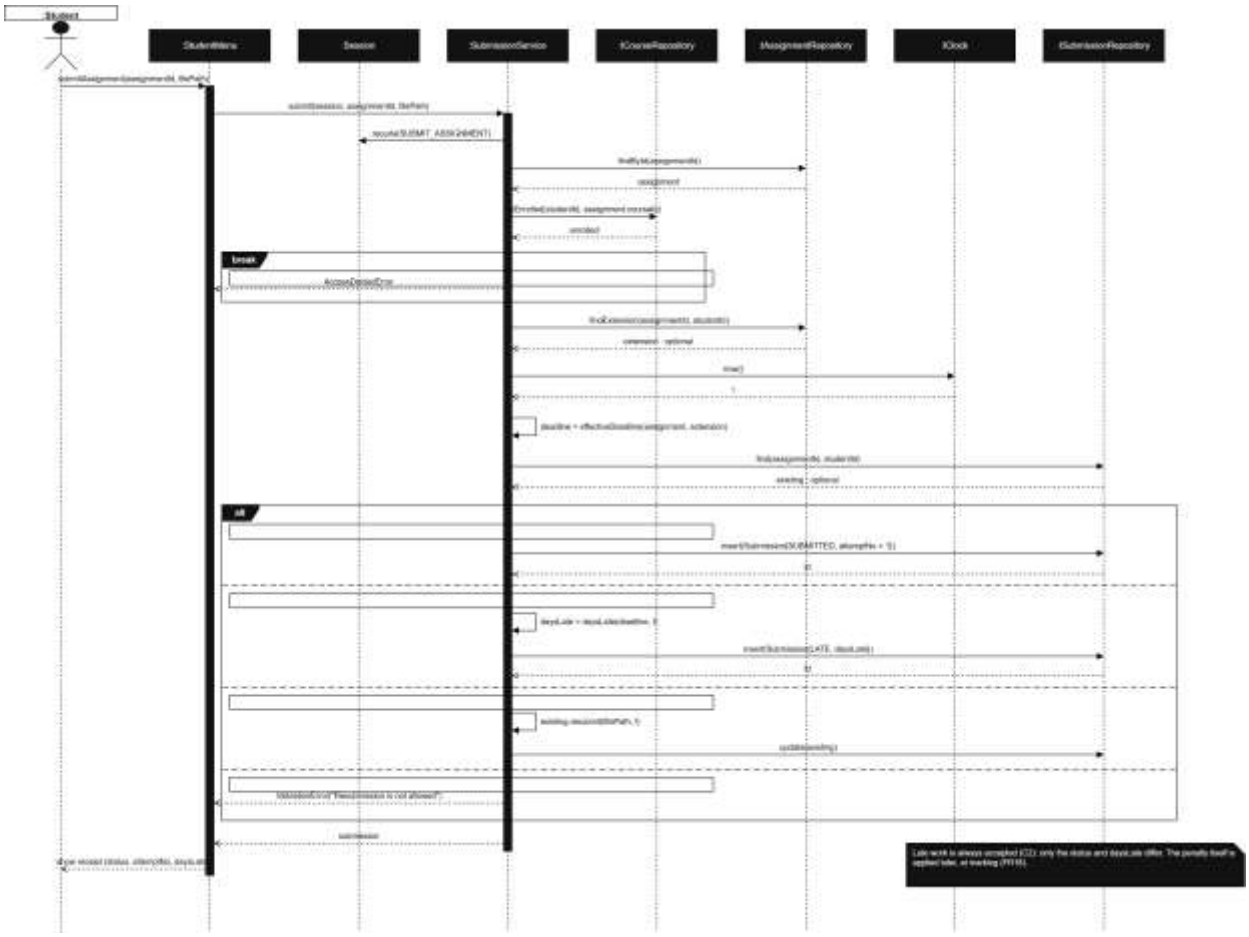
Uses four swimlanes (Student, LMS, Lecturer, Head of Department) to show the complete process. The resubmission loop exists only on the on-time path, which reflects assumption O2. After marking, a **fork** splits the flow: the student can read the written feedback straight away, while the mark continues through moderation in parallel. This is the C1 compromise drawn as behaviour. The feedback branch ends in a flow-final node because it does not need to wait for the other branch.



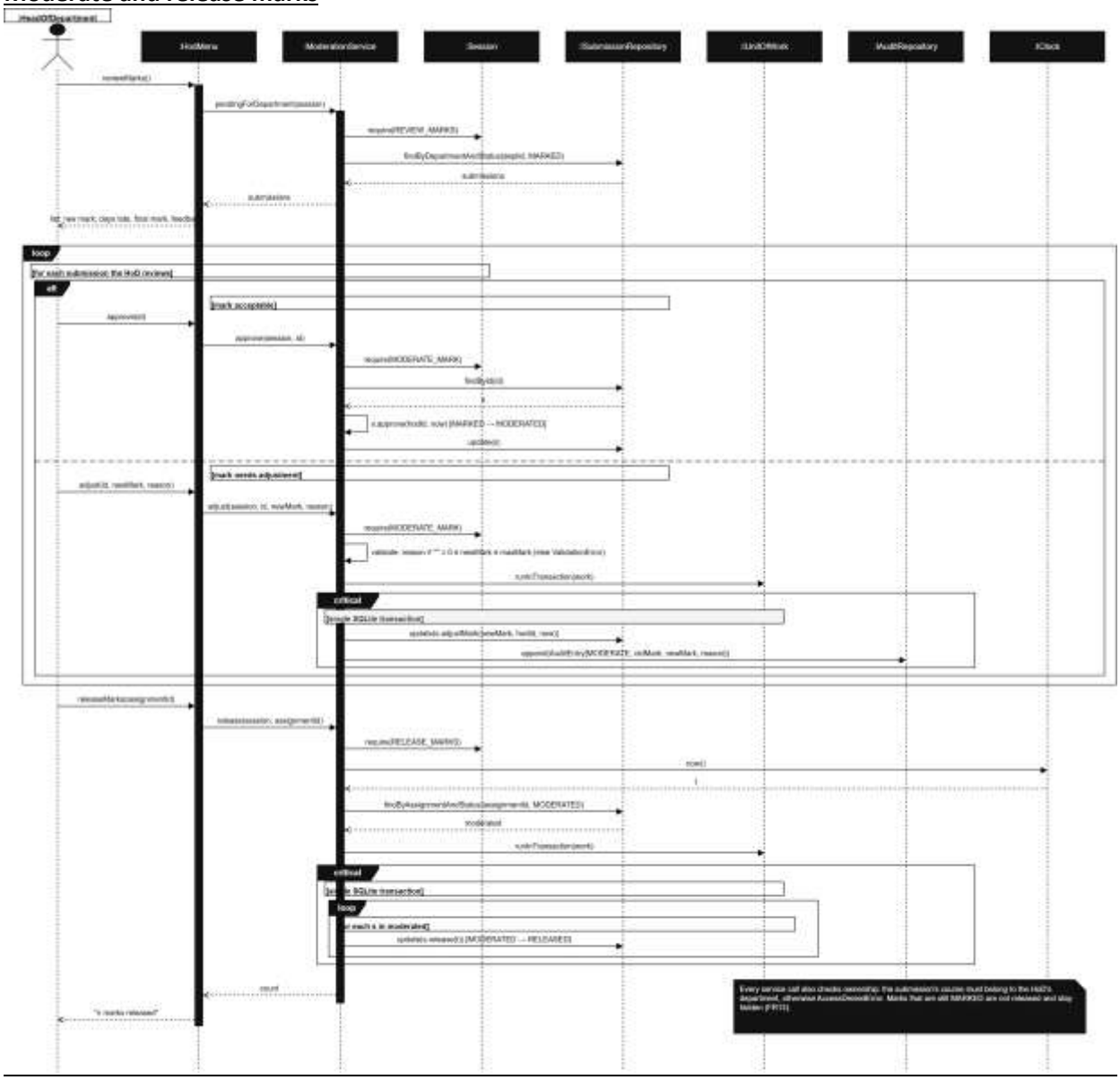
Sequence Diagram:

Submit Assignment

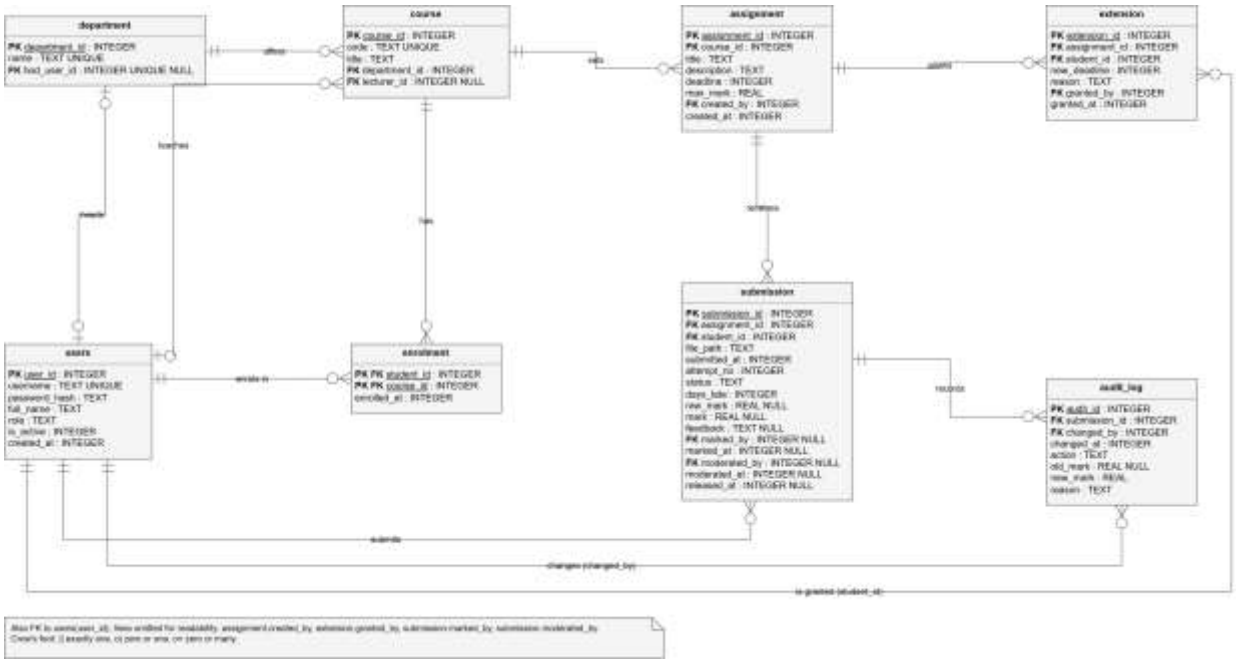
Follows the "Submit assignment" use case. The **break** fragment covers a student who is not enrolled on the course. The effective deadline is the individual extension if one exists, otherwise the assignment deadline (FR06). The **alt** fragment holds the four outcomes: an on-time first submission, a late first submission, an on-time resubmission (FR17), and a refused resubmission (late work or work already marked, O2). daysLate counts any part of a day as a whole day, so work one second late is one day late;


Student
StudentMenu
Session
SubmissionService
ICourseRepository
IAssignmentRepository
IClock
ISubmissionRepository
break
alt
Late work is always accepted (CC); only the notice and keyLate differ. The penalty itself is applied later, at marking (RF18).

Moderate and release marks



Database Design



Data dictionary

Users

Column	Type	Null	Key / constraint	Description
user_id	INTEGER	No	PK, autoincrement	Unique identifier of every user
username	TEXT	No	UNIQUE, 3-32 characters	Login name; its unique index makes login lookups fast (NFR01)
password_hash	TEXT	No	CHECK LIKE '$argon2id$%'	Argon2id hash with its salt; plain text is rejected by the database (NFR04)
full_name	TEXT	No	not empty	Display name
role	TEXT	No	CHECK IN (STUDENT, LECTURER, HOD, ADMIN)	Discriminator for the User subclass
is_active	INTEGER	No	CHECK IN (0,1), default 1	Deactivated users cannot log in (FR01)
created_at	INTEGER	No		Unix time (UTC) the account was created

Department

Column	Type	Null	Key / constraint	Description
department_id	INTEGER	No	PK	Unique identifier
name	TEXT	No	UNIQUE	Department name
hod_user_id	INTEGER	Yes	FK users, UNIQUE	The HoD; UNIQUE means one person heads at most one department

Course

Column	Type	Null	Key / constraint	Description
course_id	INTEGER	No	PK	Unique identifier
code	TEXT	No	UNIQUE	Course code, e.g. CS101
title	TEXT	No	not empty	Course title
department_id	INTEGER	No	FK department	Owning department (used for HoD scope, FR10)
lecturer_id	INTEGER	Yes	FK users	Assigned lecturer; NULL until assigned (FR03)

Enrolment

Column	Type	Null	Key / constraint	Description
student_id	INTEGER	No	PK part, FK users	Enrolled student
course_id	INTEGER	No	PK part, FK course	Course enrolled on (FR04)
enrolled_at	INTEGER	No		Unix time of enrolment

Assignment

Column	Type	Null	Key / constraint	Description
assignment_id	INTEGER	No	PK	Unique identifier
course_id	INTEGER	No	FK course	Course the assignment belongs to
title	TEXT	No	not empty	Assignment title
description	TEXT	No	default ''	Instructions
deadline	INTEGER	No		Original deadline, Unix time (UTC)
max_mark	REAL	No	CHECK > 0	Maximum mark (FR05, FR07)
created_by	INTEGER	No	FK users	Lecturer who created it
created_at	INTEGER	No		Creation time

Extension

Column	Type	Null	Key / constraint	Description
extension_id	INTEGER	No	PK	Unique identifier
assignment_id	INTEGER	No	FK assignment; UNIQUE with student_id	Assignment being extended
student_id	INTEGER	No	FK users	Student receiving the extension (individual only, C3)
new_deadline	INTEGER	No	must be later than the deadline (checked in the service)	Individual effective deadline (FR06)
reason	TEXT	No	not blank	Why the extension was granted
granted_by	INTEGER	No	FK users	Lecturer who granted it
granted_at	INTEGER	No		When it was granted

Submission

Column	Type	Null	Key / constraint	Description
submission_id	INTEGER	No	PK	Unique identifier
assignment_id	INTEGER	No	FK assignment; UNIQUE with student_id	One row per student per assignment; resubmission overwrites it (FR17)
student_id	INTEGER	No	FK users	Submitting student
file_path	TEXT	No	not empty	Location of the submitted file
submitted_at	INTEGER	No		Time of the latest (re)submission
attempt_no	INTEGER	No	CHECK >= 1, default 1	Number of submissions made
status	TEXT	No	CHECK IN (5 states)	Life-cycle state (Figure 5.5)
days_late	INTEGER	No	CHECK >= 0; > 0 exactly when LATE	Days late, frozen at submission
raw_mark	REAL	Yes	CHECK >= 0	Lecturer's mark before any penalty
mark	REAL	Yes	CHECK >= 0; required once marked	Mark after penalty and moderation; visible only when RELEASED
feedback	TEXT	Yes		Written feedback (FR08, FR09)
marked_by / marked_at	INTEGER	Yes	FK users; required once marked	Who marked it and when
moderated_by / moderated_at	INTEGER	Yes	FK users; only in MODERATED or RELEASED	HoD who moderated it and when
released_at	INTEGER	Yes	required exactly when RELEASED	Release time (FR12)

Audit_log

Column	Type	Null	Key / constraint	Description
audit_id	INTEGER	No	PK	Unique identifier
submission_id	INTEGER	No	FK submission	Submission whose mark changed
changed_by	INTEGER	No	FK users	Who changed it (FR22)
changed_at	INTEGER	No	indexed	When it was changed
action	TEXT	No	CHECK IN (MARK, REMARK, MODERATE)	Kind of change
old_mark	REAL	Yes		Previous mark (NULL for the first mark)
new_mark	REAL	No	CHECK >= 0	New mark
reason	TEXT	No	not blank	Why the mark changed

student_result_view is a read-only view. It returns feedback once a submission is marked, but returns the mark only when the status is RELEASED. The student menu reads results only through this view, which gives FR13 a second safeguard at the database level.

SQL schema

The full schema is listed below. It was run in SQLite 3.45 and checked with a test script of 42 checks:

- 23 valid operations went through the whole life cycle, and the view returned the correct values at each step;
- 19 invalid operations were rejected, including a plain-text password, an unknown status, a LATE row with zero days late, a RELEASED row without a release time, and any UPDATE or DELETE on audit_log (blocked by triggers);
- with 800 student accounts stored, the login lookup used the username index (NFR01).

```
-- ======================================================================
-- Moderated LMS - SQLite schema (v1.0)
-- Timestamps are stored as INTEGER Unix epoch seconds (UTC).
-- Enumerations are stored as TEXT and restricted with CHECK constraints.
-- ======================================================================
PRAGMA foreign_keys = ON;


-- ----------------------------------------------------------------------
-- users : one table for the whole User hierarchy (single-table inheritance)
-- ----------------------------------------------------------------------
CREATE TABLE users (
   user_id       INTEGER PRIMARY KEY AUTOINCREMENT,
   username      TEXT    NOT NULL UNIQUE CHECK (length(username) BETWEEN 3 AND 32),
   password_hash  TEXT    NOT NULL CHECK (password_hash LIKE '$argon2id$%'),
   full_name     TEXT    NOT NULL CHECK (length(full_name) > 0),
   role          TEXT    NOT NULL CHECK (role IN ('STUDENT','LECTURER','HOD','ADMIN')),
   is_active     INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0,1)),
   created_at    INTEGER NOT NULL
);

-- ----------------------------------------------------------------------
-- department
-- ----------------------------------------------------------------------
CREATE TABLE department (
   department_id  INTEGER PRIMARY KEY AUTOINCREMENT,
   name          TEXT    NOT NULL UNIQUE CHECK (length(name) > 0),
   hod_user_id    INTEGER UNIQUE REFERENCES users(user_id) ON DELETE RESTRICT
);

-- ----------------------------------------------------------------------
-- course
-- ----------------------------------------------------------------------
CREATE TABLE course (
   course_id     INTEGER PRIMARY KEY AUTOINCREMENT,
   code          TEXT    NOT NULL UNIQUE CHECK (length(code) > 0),
   title         TEXT    NOT NULL CHECK (length(title) > 0),
   department_id  INTEGER NOT NULL REFERENCES department(department_id) ON DELETE RESTRICT,
   lecturer_id    INTEGER REFERENCES users(user_id) ON DELETE RESTRICT
);

-- ----------------------------------------------------------------------
-- enrolment : resolves the Student *..* Course relationship
-- ----------------------------------------------------------------------
CREATE TABLE enrolment (
   student_id     INTEGER NOT NULL REFERENCES users(user_id)    ON DELETE RESTRICT,
   course_id      INTEGER NOT NULL REFERENCES course(course_id) ON DELETE RESTRICT,
   enrolled_at    INTEGER NOT NULL,
   PRIMARY KEY (student_id, course_id)
);

-- ----------------------------------------------------------------------
-- assignment
-- ----------------------------------------------------------------------
CREATE TABLE assignment (
   assignment_id  INTEGER PRIMARY KEY AUTOINCREMENT,
   course_id      INTEGER NOT NULL REFERENCES course(course_id) ON DELETE RESTRICT,
   title         TEXT    NOT NULL CHECK (length(title) > 0),
   description    TEXT    NOT NULL DEFAULT '',
   deadline      INTEGER NOT NULL,
   max_mark       REAL    NOT NULL CHECK (max_mark > 0),
   created_by     INTEGER NOT NULL REFERENCES users(user_id) ON DELETE RESTRICT,
   created_at     INTEGER NOT NULL
);


-- ----------------------------------------------------------------------
```

```
-- extension : at most one individual extension per student per assignment
-- ---------------------------------------------------------------------------
CREATE TABLE extension (
   extension_id   INTEGER PRIMARY KEY AUTOINCREMENT,
   assignment_id  INTEGER NOT NULL REFERENCES assignment(assignment_id) ON DELETE RESTRICT,
   student_id     INTEGER NOT NULL REFERENCES users(user_id)          ON DELETE RESTRICT,
   new_deadline   INTEGER NOT NULL,
   reason         TEXT    NOT NULL CHECK (length(trim(reason)) > 0),
   granted_by     INTEGER NOT NULL REFERENCES users(user_id) ON DELETE RESTRICT,
   granted_at     INTEGER NOT NULL,
   UNIQUE (assignment_id, student_id)
);

-- ---------------------------------------------------------------------------
-- submission : one row per student per assignment; resubmission overwrites
-- ---------------------------------------------------------------------------
CREATE TABLE submission (
   submission_id  INTEGER PRIMARY KEY AUTOINCREMENT,
   assignment_id  INTEGER NOT NULL REFERENCES assignment(assignment_id) ON DELETE RESTRICT,
   student_id     INTEGER NOT NULL REFERENCES users(user_id)          ON DELETE RESTRICT,
   file_path      TEXT    NOT NULL CHECK (length(file_path) > 0),
   submitted_at   INTEGER NOT NULL,
   attempt_no     INTEGER NOT NULL DEFAULT 1 CHECK (attempt_no >= 1),
   status         TEXT    NOT NULL CHECK (status IN ('SUBMITTED','LATE','MARKED','MODERATED','RELEASED')),
   days_late      INTEGER NOT NULL DEFAULT 0 CHECK (days_late >= 0),
   raw_mark       REAL    CHECK (raw_mark IS NULL OR raw_mark >= 0),
   mark           REAL    CHECK (mark     IS NULL OR mark     >= 0),
   feedback       TEXT,
   marked_by      INTEGER REFERENCES users(user_id) ON DELETE RESTRICT,
   marked_at      INTEGER,
   moderated_by   INTEGER REFERENCES users(user_id) ON DELETE RESTRICT,
   moderated_at   INTEGER,
   released_at    INTEGER,
   UNIQUE (assignment_id, student_id),
   -- a late status must carry a lateness value, an on-time one must not
   CHECK ((status = 'LATE') = (days_late > 0) OR status IN ('MARKED','MODERATED','RELEASED')),
   -- once marked, a mark, a marker and a timestamp must exist
   CHECK (status IN ('SUBMITTED','LATE')
         OR (raw_mark IS NOT NULL AND mark IS NOT NULL AND marked_by IS NOT NULL AND marked_at IS NOT NULL)),
   -- moderation and release data only in the matching states
   CHECK (status IN ('MODERATED','RELEASED') OR (moderated_by IS NULL AND moderated_at IS NULL)),
   CHECK (status = 'RELEASED' OR released_at IS NULL),
   CHECK (status <> 'RELEASED' OR released_at IS NOT NULL)
);

-- ---------------------------------------------------------------------------
-- audit_log : append-only record of every mark change (FR22)
-- ---------------------------------------------------------------------------
CREATE TABLE audit_log (
   audit_id       INTEGER PRIMARY KEY AUTOINCREMENT,
   submission_id  INTEGER NOT NULL REFERENCES submission(submission_id) ON DELETE RESTRICT,
   changed_by     INTEGER NOT NULL REFERENCES users(user_id)          ON DELETE RESTRICT,
   changed_at     INTEGER NOT NULL,
   action         TEXT    NOT NULL CHECK (action IN ('MARK','REMARK','MODERATE')),
   old_mark       REAL,
   new_mark       REAL    NOT NULL CHECK (new_mark >= 0),
   reason         TEXT    NOT NULL CHECK (length(trim(reason)) > 0)
);

CREATE TRIGGER audit_log_no_update BEFORE UPDATE ON audit_log
BEGIN SELECT RAISE(ABORT, 'audit_log is append-only'); END;

CREATE TRIGGER audit_log_no_delete BEFORE DELETE ON audit_log
BEGIN SELECT RAISE(ABORT, 'audit_log is append-only'); END;

-- ---------------------------------------------------------------------------
-- Indexes supporting NFR01 (login) and NFR05 (audit queries)
-- (users.username already has an index through its UNIQUE constraint)
-- ---------------------------------------------------------------------------
CREATE INDEX idx_submission_status     ON submission(status);
CREATE INDEX idx_submission_student    ON submission(student_id);
CREATE INDEX idx_assignment_course     ON assignment(course_id);
CREATE INDEX idx_course_department     ON course(department_id);
CREATE INDEX idx_audit_submission      ON audit_log(submission_id);
CREATE INDEX idx_audit_changed_at      ON audit_log(changed_at);

-- ---------------------------------------------------------------------------
-- student_result_view : the only path the Student menu uses to read results.
-- Feedback appears once marked (FR09); the mark only once released (FR13).
-- ---------------------------------------------------------------------------
CREATE VIEW student_result_view AS
SELECT s.submission_id,
```

```
    s.student_id,
    a.assignment_id,
    a.title                                 AS assignment_title,
    a.max_mark,
    s.status,
    s.days_late,
    CASE WHEN s.status IN ('MARKED','MODERATED','RELEASED')
        THEN s.feedback END                         AS feedback,
    CASE WHEN s.status = 'RELEASED' THEN s.mark END         AS mark
FROM submission s
JOIN assignment a ON a.assignment_id = s.assignment_id
```

Mapping between classes and tables

Class(es)	Table / view	Mapping and justified differences
User, Student, Lecturer, HeadOfDepartment, Administrator	users	Single-table inheritance: the role column records which subclass a row is, and UserFactory builds the correct object. The subclasses add behaviour, not data, so a table per class would only add joins to every login. That alternative (class-table inheritance) was rejected for NFR01.
HeadOfDepartment.departmentId	department.hod_user_id	Stored on the department side, so UNIQUE can guarantee that a department has at most one HoD and a HoD heads at most one department. The repository fills departmentId with a join.
Department, Course, Assignment, Extension, AuditEntry	same-named tables	One-to-one: each attribute maps to a column of the same meaning.
Enrolment	enrolment	Association class with a composite primary key.
Submission	submission	SubmissionStatus maps to TEXT with a CHECK; each std::optional maps to a nullable column. The state rules are enforced both in Submission::transitionTo and by CHECK constraints.
StudentResult	student_result_view	Read model for the student menu; hides the mark until release (FR13).
Timestamp	INTEGER	Unix seconds (UTC): easy to compare against deadlines and unaffected by time zones.
Enumerations (Role, SubmissionStatus, AuditAction)	TEXT + CHECK	Readable in the database and still restricted to valid values.
Services, strategies, factories, Session	none	Behaviour only; nothing to store.